

Research Databases and Market Data Storage

Design durable, point-in-time, query-efficient, recoverable quantitative data stores.

Library of the Quant

Educational reference manual

Manual Profile

LIBRARY ID	LOTQ-0046
CATEGORY	Data Engineering
DIFFICULTY	Intermediate
FORMAT	Single-column field-guide manual
USE	Study, lookup, and research-system reference

Educational Use Only

This manual is educational material for quant research study. It is not financial advice, not a trading recommendation, and not a guarantee of correctness or live trading usefulness.

Table of contents

INTERMEDIATE

ORIENTATION	
Title	1
How to use this manual	5
Notation and storage contract	6
Reference market-data storage workflow	7
STORAGE FOUNDATIONS	
Workload-first storage design	8
Row, columnar, and hybrid engines	9
OLTP and OLAP boundaries	10
Lake, warehouse, and lakehouse patterns	11
Raw, canonical, and serving layers	12
Immutability, snapshots, and versions	13
DATA MODELS AND LAYOUT	
Relational normalization and reference data	14
Long, wide, and star schemas	15
Event logs, bars, and state tables	16

Table of contents

INTERMEDIATE

DATA MODELS AND LAYOUT - CONTINUED	
Keys, identity, and symbol history	17
Schema evolution and type discipline	18
Partitioning, sorting, and clustering	19
Indexes and data skipping	20
QUERY AND PERFORMANCE	
Columnar formats, encodings, and compression	21
Query plans, pruning, and pushdown	22
Point-in-time and as-of querying	23
Materialized views, caches, and projections	24
Concurrency and workload management	25
GOVERNANCE AND OPERATIONS	
Transactions and atomic publication	26
Quality constraints and reconciliation	27

Table of contents

INTERMEDIATE

GOVERNANCE AND OPERATIONS - CONTINUED

Backups, recovery, and disaster readiness	28
Retention, tiering, compaction, and vacuum	29
Security, entitlements, and data licensing	30

VISUAL LABS AND REVIEW

Storage architecture and truth boundaries	31
Choose storage from the workload	32
Worked grain and key design	33
Storage and query cost model	34
Partition strategies by workload	35
Point-in-time query anatomy	36
Compression ratio versus decode speed	37
Read a query plan from the leaves	38
Atomic publication and crash recovery	39
Recovery objectives by data class	40
Entitlement propagation	41
Safe storage migration	42
Storage failure diagnostics	43
Database observability scorecard	44
Storage design promotion checklist	45
Final active recall and system index	46

How to use this manual

INTERMEDIATE

1. BEGIN WITH THE CONSUMER

Name the decision, query shape, data horizon, freshness deadline, concurrency, and failure tolerance. Storage choices become defensible only when tied to this workload contract.

2. DECLARE ONE ROW

Write the grain, stable key, source key, units, clocks, revision rule, and null semantics. Do this before choosing indexes, partitions, or table shapes.

3. SEPARATE EVIDENCE FROM ACCELERATION

Preserve immutable raw and canonical truth, then build serving projections, materialized views, and caches with explicit parent snapshots. Performance layers must remain replaceable.

4. READ QUERY PLANS

Inspect bytes scanned, rows surviving, join cardinality, shuffle, spill, skew, and tail latency. A correct SQL result can still violate operational constraints.

5. REHEARSE CHANGE

Test schema evolution, late corrections, concurrent writers, compaction, snapshot rollback, backup restoration, and entitlement changes. Record which consumers are affected.

6. MEASURE LIFECYCLE COST

Forecast retained bytes, object count, compute, egress, maintenance, and restore time. Cost policy should preserve reproducibility and contracts, not merely delete old files.

7. TIE CONTROLS TO CAPITAL

Map freshness, quality, security, and recovery failures to features, models, backtests, portfolios, and reports. Define stop, degraded, and rollback behavior before incidents.

DECISION STANDARD

Approve a storage design only when its truth boundary, point-in-time semantics, performance envelope, recovery path, and entitlement policy can all be demonstrated with evidence.

Notation and storage contract

INTERMEDIATE

SYMBOL	OBJECT	DATABASE MEANING
G	row grain	Economic meaning represented by one logical record
K	stable key	Fields that uniquely identify a row under revision policy
S_v	table snapshot	Complete immutable table state at version v
t_e	event time	When the observed market or business event occurred
t_a	availability time	When the record became eligible for a consumer
P	partition spec	Coarse physical grouping used for pruning and repair
A_r	read amplification	Bytes scanned divided by bytes needed
R_c	compression ratio	Uncompressed bytes divided by compressed bytes
RPO	recovery point	Maximum acceptable interval of unrecoverable data
RTO	recovery time	Maximum acceptable service restoration interval

READING RULE

A stored value is incomplete without row identity, physical units, information clocks, quality state, snapshot identity, and entitlement. Query speed never compensates for ambiguity in any of these fields.

Reference market-data storage workflow

INTERMEDIATE

1

Profile workloads

Consumers, predicates, history, freshness, revisions, concurrency, recovery

2

Capture raw evidence

Immutable payload, source identity, sequence, clocks, license and checksum

3

Canonicalize semantics

Stable keys, grain, types, units, calendars, revisions and quality

4

Choose table model

Events, state, facts, dimensions, long or wide consumer representation

5

Design physical layout

Partitions, clustering, row groups, indexes, compression and file size

6

Publish snapshots

Atomic manifest, schema, parents, checksums, readiness and entitlements

7

Serve and observe

Plans, latency, bytes, freshness, cache identity and consumer impact

8

Maintain and recover

Compaction, retention, backup, restore drills, migration and audit

BOUNDARY RULE

Raw evidence remains immutable, canonical tables expose governed snapshots, and serving layers declare freshness and parents. Consumers pin versions so maintenance and corrections cannot silently rewrite approved research.

Workload-first storage design

INTERMEDIATE

ONE-SENTENCE MEANING

Storage design begins with the decisions and access patterns the system must support, not with a favorite database product. The same market data may need an immutable archive for replay, a scan-optimized analytical copy, and a low-latency serving view.

MEMORY JOGGER

Start with five questions: what is one row, which time range is read, how fresh must it be, how often is it revised, and what failure can the consumer tolerate? Answers determine layout, indexing, consistency, and retention more reliably than vendor feature lists.

WHY IT MATTERS IN QUANT RESEARCH

Quant workloads range from full-universe cross-sectional scans to single-symbol event replay and live point lookups. A store optimized for one pattern can be expensive or dangerously slow for another, so architecture should separate evidence preservation from consumer-specific acceleration.

FORMULA INTUITION

A useful cost model is $\text{total cost} = \text{storage bytes} \times \text{retention price} + \text{bytes scanned} \times \text{query price} + \text{write and metadata overhead} + \text{operational labor}$. Latency should be measured at the relevant percentile and data volume, because average response time hides tail failures near decision cutoffs.

SIMPLE MARKET EXAMPLE

A nightly factor job scans ten years across 8,000 securities, while a live risk check fetches the latest position and quote for one account. The first prefers large columnar partitions; the second needs indexed or in-memory keys with strict freshness.

PYTHON / SYSTEM IMPLEMENTATION

Capture query templates, filters, row counts, bytes read, latency percentiles, freshness deadlines, and concurrency before choosing a physical design. Re-run representative benchmarks whenever universe breadth, history, or data frequency changes materially.

CONFUSIONS TO AVOID

Do not call a database scalable because it handled a toy sample or a single analyst. Scale includes concurrent readers, backfills, retention, small-file pressure, recovery, and the worst credible market session.

WHERE THIS FITS IN THE RESEARCH SYSTEM

This analysis belongs before ingestion and schema implementation. It becomes the nonfunctional contract used by storage, pipeline, research, backtest, and live-serving teams.

RELATED CONCEPTS AND QUICK RECALL

Related: workload profile, service-level objective, benchmark, access pattern. Recall task: Specify the dominant read, write, revision, latency, and recovery requirements for one research dataset.

Row, columnar, and hybrid engines

INTERMEDIATE

ONE-SENTENCE MEANING

Row stores keep the fields of a record together, while columnar stores keep values of the same field together for compression and selective scans. Hybrid systems combine transaction-friendly ingestion with analytical replicas or columnar segments, but the consistency boundary

MEMORY JOGGER

Choose row orientation for narrow keyed reads and frequent updates; choose column orientation for large scans over a subset of fields. A hybrid is useful when one operational truth must feed both patterns without forcing either workload onto the wrong physical shape.

WHY IT MATTERS IN QUANT RESEARCH

Market research repeatedly aggregates returns, volume, spread, and attributes across long histories, which favors columnar execution. Order state, job metadata, entitlements, and experiment records often need atomic point updates, which favor row-oriented transactions.

FORMULA INTUITION

Approximate scan work as selected columns divided by total columns times compressed partition bytes, adjusted for pruning. Row lookup cost is dominated by index depth and page access, whereas columnar cost is dominated by decompression, vectorized operators, and bytes surviving filters.

SIMPLE MARKET EXAMPLE

A 100-column quote table queried for timestamp, instrument, bid, and ask may read a small fraction of a columnar file. The same query against a row heap can touch every field, while a keyed latest-quote lookup may still favor a row index.

PYTHON / SYSTEM IMPLEMENTATION

Benchmark actual predicates in both cold-cache and warm-cache conditions, then record compression ratio and CPU spent decoding. Use a change-data or batch replication contract when operational rows feed an analytical copy.

CONFUSIONS TO AVOID

Columnar does not mean fast for every query, and row-oriented does not mean unsuitable for history. Tiny scans, poor clustering, excessive files, or selective indexes can reverse simplistic expectations.

WHERE THIS FITS IN THE RESEARCH SYSTEM

The canonical archive and research warehouse usually emphasize columnar history; operational catalogs and registries emphasize transactions. Serving layers may add indexed projections without becoming the authoritative evidence store.

RELATED CONCEPTS AND QUICK RECALL

Related: row store, column store, vectorized execution, analytical replica. Recall task: Explain why the same quote data might have both a columnar history and a keyed serving projection.

OLTP and OLAP boundaries

INTERMEDIATE

ONE-SENTENCE MEANING

Online transaction processing protects small state changes with concurrency control, while online analytical processing scans and aggregates large historical sets. Separating these workloads prevents research queries from blocking operational writes and prevents

MEMORY JOGGER

OLTP answers what is the current committed state; OLAP answers how did populations behave across time. The boundary is a governed replication or publication step with clocks, versions, and reconciliation.

WHY IT MATTERS IN QUANT RESEARCH

Experiment metadata, model approvals, schedules, and user permissions need transactional integrity. Backtests, feature diagnostics, attribution, and cross-sectional statistics need broad scans that should not contend with those operational tables.

FORMULA INTUITION

Transaction throughput is often described in committed operations per second under a contention profile. Analytical throughput is better described by scanned bytes, rows processed, concurrent queries, spill, and time to a decision-ready result.

SIMPLE MARKET EXAMPLE

A researcher joining a billion-row trade table to a production order database can create locks, unpredictable latency, and accidental access to mutable state. Publishing order outcomes into a point-in-time analytical table preserves both performance and meaning.

PYTHON / SYSTEM IMPLEMENTATION

Define source-of-truth ownership, replication lag, change ordering, deletion semantics, and reconciliation totals. Consumers should see an explicit data-ready watermark rather than infer completeness from a table being reachable.

CONFUSIONS TO AVOID

A read replica is not automatically an analytical system if its schema, indexing, and storage orientation remain transaction-centric. Conversely, a warehouse should not be used to coordinate low-latency mutable workflows merely because it supports updates.

WHERE THIS FITS IN THE RESEARCH SYSTEM

The transactional catalog records jobs, artifacts, approvals, and access state; the analytical layer receives immutable or versioned facts. Lineage connects the two without merging their service objectives.

RELATED CONCEPTS AND QUICK RECALL

Related: OLTP, OLAP, change data capture, read replica. Recall task: Draw the handoff from a transactional experiment registry to an analytical research dataset.

Lake, warehouse, and lakehouse patterns

INTERMEDIATE

ONE-SENTENCE MEANING

A data lake preserves files in object storage, a warehouse provides governed analytical tables and managed execution, and a lakehouse adds table metadata and transactional behavior over lake storage. These are architectural patterns with overlapping implementations rather

MEMORY JOGGER

Separate the bytes, the table contract, and the compute engine. A lakehouse table is more than a directory of Parquet files because snapshots, schema, deletion, and concurrency are managed through durable metadata.

WHY IT MATTERS IN QUANT RESEARCH

Quant teams need inexpensive historical retention, reproducible snapshots, point-in-time query behavior, and efficient scans. The right pattern depends on data volume, concurrency, governance, operational skill, and whether multiple engines must read the same evidence.

FORMULA INTUITION

Effective cost includes stored bytes, metadata operations, compute, egress, maintenance, and engineer time. Snapshot correctness depends on an atomic table version that binds a complete set of data and delete files rather than a best-effort directory listing.

SIMPLE MARKET EXAMPLE

Raw exchange packets may remain in an object-store lake, canonical bars may live in a transactional table format, and curated risk aggregates may be published into a managed warehouse. Each layer serves a distinct contract and retention policy.

PYTHON / SYSTEM IMPLEMENTATION

Register every logical table with schema, partition spec, snapshot ID, owner, clock fields, and allowed engines. Test concurrent writes, snapshot reads, compaction, rollback, and metadata recovery before adopting lakehouse semantics.

CONFUSIONS TO AVOID

Parquet alone does not provide transactions, and a warehouse export is not a durable raw archive by default. Product categories should never replace an explicit statement of consistency, snapshot, and recovery behavior.

WHERE THIS FITS IN THE RESEARCH SYSTEM

The lake holds immutable source evidence, the governed table layer supplies canonical history, and the warehouse or query service supplies curated access. Research artifacts pin snapshots instead of relying on a moving latest view.

RELATED CONCEPTS AND QUICK RECALL

Related: object storage, warehouse, table format, snapshot isolation. Recall task: State which layer should hold raw packets, canonical bars, and curated factor aggregates, and why.

Raw, canonical, and serving layers

INTERMEDIATE

ONE-SENTENCE MEANING

A raw layer preserves received evidence, a canonical layer resolves identity and semantics, and a serving layer reshapes governed data for a specific consumer. The layers should differ by contract and responsibility, not merely by folder name.

MEMORY JOGGER

Raw answers what arrived, canonical answers what it means, and serving answers how a consumer can retrieve it efficiently. Corrections create new versions downstream while the original evidence remains available for audit and replay.

WHY IT MATTERS IN QUANT RESEARCH

Research fails when vendor-specific fields leak into models, adjusted and unadjusted prices mix, or current reference mappings rewrite history. Layered storage isolates source quirks, centralizes semantic policy, and permits consumer acceleration without losing lineage.

FORMULA INTUITION

Let R be immutable source records, $C = \text{canonicalize}(R, \text{policy})$, and $S_j = \text{project}(C, \text{consumer}_j)$. Reproducibility requires each serving snapshot to reference the exact canonical snapshot and policy version used to derive it.

SIMPLE MARKET EXAMPLE

A raw trade record retains vendor code and payload, the canonical trade resolves stable instrument and normalized units, and a one-minute serving table clusters by session and instrument. The serving table is replaceable because lineage points back to canonical truth.

PYTHON / SYSTEM IMPLEMENTATION

Use separate namespaces and permissions, enforce one-way lineage, and prevent consumers from mutating canonical tables. Publish quality reports and clock frontiers with each layer so availability is machine-readable.

CONFUSIONS TO AVOID

Bronze, silver, and gold labels do not guarantee good architecture; a so-called gold table can still contain ambiguous grain or future information. Also avoid treating serving projections as disposable if consumers cannot reconstruct their exact historical version.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Ingestion owns raw evidence, data modeling owns canonical state, and application teams own serving projections. The artifact registry connects all versions used by features, models, backtests, and live decisions.

RELATED CONCEPTS AND QUICK RECALL

Related: raw zone, canonical table, serving view, lineage. Recall task: Trace one adjusted close from vendor payload through canonical state into a factor-research query.

Immutability, snapshots, and versions

INTERMEDIATE

ONE-SENTENCE MEANING

Immutability means published evidence is not changed in place, while snapshots bind a consistent table state and versions provide durable identity across revisions. Together they allow an old backtest to be reproduced after corrections and schema changes.

MEMORY JOGGER

Never let latest be the only address. Mutable aliases are convenient discovery pointers, but research and approvals must pin immutable snapshot or artifact identifiers.

WHY IT MATTERS IN QUANT RESEARCH

Vendors correct ticks, corporate actions are revised, and data policies evolve. Versioned storage lets the team compare what changed, scope affected research, and preserve the historical information set behind a decision.

FORMULA INTUITION

A snapshot can be modeled as $M_v = \{\text{data files, delete files, schema, partition spec, parents, commit clock}\}$. Its logical checksum should change whenever any behaviorally relevant component changes, even if most physical files are reused.

SIMPLE MARKET EXAMPLE

A late correction replaces one quote partition in a new snapshot while 99.9% of files remain shared. The original strategy result still references the prior snapshot, and an impact run can compare the two versions explicitly.

PYTHON / SYSTEM IMPLEMENTATION

Use immutable object names, atomic manifest commits, snapshot retention rules, and conditional pointer updates. Record commit lineage and prevent garbage collection of files still referenced by retained artifacts.

CONFUSIONS TO AVOID

Copying files into a dated folder is not sufficient if readers can observe an incomplete set or if metadata is overwritten. Excessive version retention also has cost, so deletion must be lineage-aware rather than indefinite by habit.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Every model matrix, backtest, and production package should pin table versions. Version comparison and forward lineage drive correction analysis, rebuild plans, and audit responses.

RELATED CONCEPTS AND QUICK RECALL

Related: snapshot, manifest, time travel, immutable artifact. Recall task: Explain how a one-partition correction can create a new complete snapshot without duplicating all data.

Relational normalization and reference data

INTERMEDIATE

ONE-SENTENCE MEANING

Normalization separates facts into tables whose non-key fields depend on a stable key, reducing contradictory updates and preserving reusable reference history. Quant systems often normalize identifiers, venues, calendars, instruments, and corporate actions before

MEMORY JOGGER

Normalize when repeated attributes have independent life cycles; denormalize when a point-in-time consumer needs a stable scan shape. Keep the normalized source of semantic truth even when performance projections repeat fields.

WHY IT MATTERS IN QUANT RESEARCH

Instrument identity, listing history, classifications, and actions change on different clocks. A single wide security table encourages current values to overwrite history and makes effective-time relationships hard to validate.

FORMULA INTUITION

A functional dependency $K \rightarrow A$ means key K determines attribute A under the table's declared grain. Temporal normalization adds validity intervals and version identity so multiple historical values are valid for different effective or knowledge periods.

SIMPLE MARKET EXAMPLE

One issuer may have multiple instruments and each instrument multiple listings. Storing sector on every trade duplicates mutable reference state, while a point-in-time join can attach the appropriate sector to a research projection.

PYTHON / SYSTEM IMPLEMENTATION

Create stable surrogate or domain identifiers, enforce foreign keys where practical, and test interval overlap and orphan references. Materialize joined research tables only after applying explicit effective and availability rules.

CONFUSIONS TO AVOID

Normalization does not guarantee temporal correctness; joining a normalized current classification can still leak the future. Over-normalizing hot analytical paths can also create expensive joins without adding meaningful integrity.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Reference databases sit between raw sources and canonical market facts. Their point-in-time versions feed universe construction, corporate-action processing, feature engineering, and execution simulation.

RELATED CONCEPTS AND QUICK RECALL

Related: normal form, functional dependency, reference data, temporal table. Recall task: Model issuer, instrument, listing, and sector history without relying on ticker as identity.

Long, wide, and star schemas

INTERMEDIATE

ONE-SENTENCE MEANING

A long table stores observations as repeated dimension-value rows, a wide table places many measures in columns, and a star schema joins a central fact table to descriptive dimensions. Each shape trades flexibility, scan efficiency, sparse data handling, and semantic

MEMORY JOGGER

Use long form for extensible measures and tidy transformations, wide form for stable dense matrices, and stars for governed business facts with reusable dimensions. Preserve the row grain prominently because shape alone does not reveal meaning.

WHY IT MATTERS IN QUANT RESEARCH

Feature research frequently pivots between long observations and wide model matrices. Attribution and reporting benefit from stars, while extremely wide evolving feature tables can become fragile under schema drift.

FORMULA INTUITION

Long-form row count is approximately entities x times x populated measures, while wide-form column count grows with measures. Query cost depends on sparsity, compression, selected fields, and whether pivoting occurs once during publication or repeatedly at read time.

SIMPLE MARKET EXAMPLE

A daily feature table may store instrument-date-feature-value rows for exploration, then publish a frozen instrument-date matrix for training. A star-shaped trade fact can link instrument, venue, strategy, and calendar dimensions.

PYTHON / SYSTEM IMPLEMENTATION

Use explicit fact grain, dimension keys, measure units, and effective-date joins. Benchmark pivots, restrict uncontrolled feature-name cardinality, and version the matrix column order used by models.

CONFUSIONS TO AVOID

A wide table is not automatically denormalized safely, and long form can hide incompatible units in one value column. Star dimensions can leak current descriptions into history unless temporal validity is modeled.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Canonical observations often remain long or fact-oriented; curated training and reporting stores publish wide or star projections. The model registry records the exact schema and column order it consumed.

RELATED CONCEPTS AND QUICK RECALL

Related: tidy data, feature matrix, fact table, dimension table. Recall task: Choose long, wide, or star form for exploratory features, model training, and execution attribution.

Event logs, bars, and state tables

INTERMEDIATE

ONE-SENTENCE MEANING

An event log records changes in occurrence order, a bar aggregates events over a declared window, and a state table records the latest or interval-valid condition. Moving among them requires explicit ordering, window, revision, and completeness rules.

MEMORY JOGGER

Events say what happened, bars summarize what happened in a window, and state says what was true at a cutoff. Do not infer one representation from another unless the transformation contract is preserved.

WHY IT MATTERS IN QUANT RESEARCH

Trades and quotes support microstructure replay, bars support efficient research, and order-book or borrow state supports point-in-time decisions. Confusing event and state semantics causes duplicate counting, missed deletions, and stale values.

FORMULA INTUITION

A bar over window W can include open = first eligible price, high and low extrema, close = last eligible price, volume = sum size, plus event count and quality flags. State at cutoff t is the ordered reduction of admissible events through t , not simply the row with maximum event time in a corrected

SIMPLE MARKET EXAMPLE

A quote update changes only the bid size while leaving ask fields unchanged. A delta event must be applied to prior state, whereas treating it as a complete snapshot would null or reset valid fields.

PYTHON / SYSTEM IMPLEMENTATION

Store sequence, event and arrival clocks, event type, source key, and revision status. Build bars and state with tested reducers, checkpoint metadata, and batch-stream parity fixtures.

CONFUSIONS TO AVOID

A one-minute bar cannot reconstruct intra-minute path, spread, or event order. Latest-row queries can also select a late correction that was unavailable at the historical decision cutoff.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Raw event storage feeds canonical reducers, bar factories, book reconstruction, latency analysis, and simulation. Derived bars and states carry parent frontiers so consumers know the evidence range included.

RELATED CONCEPTS AND QUICK RECALL

Related: event sourcing, OHLCV bar, snapshot, state reducer. Recall task: Explain which fields are required to rebuild quote state and a one-minute bar from events.

Keys, identity, and symbol history

INTERMEDIATE

ONE-SENTENCE MEANING

Keys identify logical rows, while stable instrument identifiers preserve economic identity across ticker, venue, and listing changes. Market data tables need both domain identity and source-level keys because vendor corrections and duplicates rarely align with display symbols.

MEMORY JOGGER

A ticker is an attribute with a validity interval, not a permanent primary key. State one row's grain, then choose the smallest key that proves uniqueness under the revision policy.

WHY IT MATTERS IN QUANT RESEARCH

Corporate actions, migrations, cross-listings, futures rolls, and ticker reuse create false continuity or fragmentation when identity is weak. Reliable keys are required for point-in-time joins, deduplication, universe history, and PnL continuity.

FORMULA INTUITION

A composite event key may include source, venue, channel, session, sequence, and message type; a canonical observation key may include stable instrument, timestamp, field, and revision. Uniqueness is tested after explicitly selecting accepted revisions rather than across all raw records.

SIMPLE MARKET EXAMPLE

The ticker ABC can represent one issuer before a merger and another years later. Backfilling the current stable ID across both histories would fabricate a price series and invalidate factor exposures.

PYTHON / SYSTEM IMPLEMENTATION

Maintain mapping tables with valid-from, valid-to, available-from, source identifier, canonical identifier, and confidence or status. Emit unmapped and multiply mapped records to a review queue instead of choosing silently.

CONFUSIONS TO AVOID

Surrogate IDs are useful but not self-explanatory; lineage must preserve how each mapping was established. Hashing an unstable composite key can hide semantic errors rather than fix them.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Identity services feed ingestion, canonical facts, corporate actions, universe construction, features, positions, and reconciliation. Mapping versions must travel with every downstream snapshot.

RELATED CONCEPTS AND QUICK RECALL

Related: primary key, natural key, surrogate key, symbology. Recall task: Design raw and canonical keys for exchange trades and explain how ticker reuse is handled.

Schema evolution and type discipline

INTERMEDIATE

ONE-SENTENCE MEANING

Schema evolution changes fields, types, null rules, or semantics while maintaining a governed compatibility contract. Type discipline prevents timestamps, prices, currency, categorical codes, and missing states from becoming ambiguous bytes.

MEMORY JOGGER

Adding a nullable field is often physically easy but semantically incomplete. Every change needs meaning, default behavior, backfill scope, consumer compatibility, and a version boundary.

WHY IT MATTERS IN QUANT RESEARCH

Quant results can shift when decimal values become floating point, timestamp precision changes, or a missing code is reinterpreted as zero. Silent coercion creates plausible numbers that evade basic tests.

FORMULA INTUITION

Compatibility can be backward, forward, or full depending on whether old readers consume new data and new readers consume old data. Decimal precision must cover maximum notional and smallest tick; timestamp types must bind timezone and resolution.

SIMPLE MARKET EXAMPLE

A vendor changes volume from shares to lots without renaming the field. The physical integer type remains valid, but the semantic schema changed and all dependent normalization must publish a new version.

PYTHON / SYSTEM IMPLEMENTATION

Use a schema registry with field IDs, logical types, units, constraints, deprecation state, and compatibility tests. Reject unsafe narrowing, record cast failures, and run consumer contracts before publication.

CONFUSIONS TO AVOID

Column-name stability is not semantic stability, and automatic inference is unsafe for empty or unusual partitions. Nullable does not mean missing values have no policy; distinguish absent, unknown, not applicable, and invalid.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Schemas govern raw parsing, canonical tables, feature matrices, APIs, and archived snapshots. Model packages pin field order, type, unit, and preprocessing state independently of current defaults.

RELATED CONCEPTS AND QUICK RECALL

Related: schema registry, compatibility, logical type, semantic version. Recall task: Classify a unit change, field addition, timestamp precision change, and type narrowing by compatibility risk.

Partitioning, sorting, and clustering

INTERMEDIATE

ONE-SENTENCE MEANING

Partitioning creates coarse addressable groups, while sorting and clustering arrange values within or across those groups to improve pruning and locality. A good design follows common filters and repair boundaries without generating excessive files or skew.

MEMORY JOGGER

Partition coarsely enough to keep metadata manageable, then cluster on selective dimensions used inside those partitions. Let measured queries and backfills determine the design rather than copying a generic date-symbol recipe.

WHY IT MATTERS IN QUANT RESEARCH

Time-range scans, cross-sectional research, and corrected-session rebuilds all depend on physical layout. Poor choices increase bytes read, open-file operations, shuffle, and the blast radius of maintenance.

FORMULA INTUITION

Read amplification equals bytes scanned divided by bytes actually needed, while metadata amplification can be approximated by objects opened per useful gigabyte. Target file size balances parallelism against request overhead and should be evaluated after compression.

SIMPLE MARKET EXAMPLE

Daily bars may partition by month and cluster by session then instrument; nanosecond quotes may partition by venue and hour with instrument-time sorting. Symbol-per-day partitioning can create millions of tiny objects and cripple cross-sectional scans.

PYTHON / SYSTEM IMPLEMENTATION

Collect predicate frequencies, partition sizes, row-group statistics, skew, file counts, and repair history. Add compaction and clustering jobs whose outputs are atomic new snapshots, not in-place rearrangements.

CONFUSIONS TO AVOID

High-cardinality partition keys and very fine time buckets cause metadata pressure even when each file reads quickly. Clustering is not permanent because new appends can degrade locality unless maintenance restores it.

WHERE THIS FITS IN THE RESEARCH SYSTEM

The table layout contract informs query engines, schedulers, storage forecasts, and backfill planners. Partition changes require snapshot migration and benchmark evidence because they alter operational behavior.

RELATED CONCEPTS AND QUICK RECALL

Related: partition pruning, clustering, row group, small-file problem. Recall task: Propose and defend layouts for daily bars, tick events, and point-in-time reference history.

Indexes and data skipping

INTERMEDIATE

ONE-SENTENCE MEANING

Indexes map search keys to record locations, while data-skipping metadata summarizes blocks so an engine can avoid reading irrelevant ranges. Both accelerate selective access but add storage, write, maintenance, and correctness obligations.

MEMORY JOGGER

Use indexes when predicates are selective and stable; use min-max, bloom, or zone metadata when files are ordered enough to exclude blocks. An index that is not maintained under revisions is worse than a slower honest scan.

WHY IT MATTERS IN QUANT RESEARCH

Research alternates between broad scans and exact lookups such as one instrument, event ID, or artifact. Purpose-built access paths can cut latency dramatically without duplicating the entire canonical dataset.

FORMULA INTUITION

Index benefit depends on selectivity s , clustering, and random-page cost; as s approaches a large share of the table, a sequential scan may win. Bloom filters trade false positives for compact membership tests and never justify false negatives.

SIMPLE MARKET EXAMPLE

A trade replay query for one instrument over one session benefits from time clustering and an instrument skip index. A market-wide daily aggregation may ignore the index because touching nearly every block is cheaper sequentially.

PYTHON / SYSTEM IMPLEMENTATION

Inspect query plans, track index hit and maintenance cost, and rebuild or drop access paths that no longer improve representative workloads. Validate skip metadata after compaction, deletes, and schema evolution.

CONFUSIONS TO AVOID

More indexes do not guarantee better performance because writes, storage, optimizer choice, and vacuum work all increase. A min-max index on randomly ordered symbols offers little pruning despite looking present in metadata.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Indexes belong to serving or query-acceleration layers unless the canonical store requires the same access pattern. Catalogs record which snapshot, columns, and maintenance policy each access path covers.

RELATED CONCEPTS AND QUICK RECALL

Related: B-tree, bloom filter, zone map, selectivity. Recall task: Choose an access path for exact event lookup, one-symbol replay, and full-universe aggregation.

Columnar formats, encodings, and compression

INTERMEDIATE

ONE-SENTENCE MEANING

Columnar file formats organize values into typed column chunks with statistics and encodings, while compression reduces physical bytes at the cost of CPU. Effective design chooses row groups, codecs, dictionaries, and sort order from actual value distributions and query

MEMORY JOGGER

Compression is part of execution, not merely archival savings. Smaller scans often run faster until decode CPU or poor row-group sizing becomes the new bottleneck.

WHY IT MATTERS IN QUANT RESEARCH

Market history contains repeated identifiers, monotonic timestamps, sparse flags, and numeric measures with different entropy. Matching encoding to each field can reduce storage and I/O without losing exactness.

FORMULA INTUITION

Compression ratio = uncompressed bytes divided by compressed bytes, and effective throughput = logical bytes processed divided by elapsed time. Dictionary encoding helps low-cardinality strings; delta encoding helps ordered integers and timestamps; run-length encoding helps long repeated

SIMPLE MARKET EXAMPLE

Sorting quotes by instrument and event time makes timestamp deltas small and instrument runs long. A random interleave weakens both compression and data skipping even if the same codec is selected.

PYTHON / SYSTEM IMPLEMENTATION

Measure compressed size, encode and decode CPU, predicate pruning, memory, and query time across representative data. Keep lossless source values, and use explicit decimal or integer scaling when exact price ticks matter.

CONFUSIONS TO AVOID

The highest compression ratio is not always the fastest or cheapest overall. Oversized row groups reduce parallelism and selective reads, while tiny groups inflate metadata and lower compression efficiency.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Canonical analytical tables standardize a durable interoperable format, and serving layers may use engine-specific encodings. Snapshot manifests record format, codec, schema, and writer version for reproducibility.

RELATED CONCEPTS AND QUICK RECALL

Related: Parquet, dictionary encoding, delta encoding, compression ratio. Recall task: Match encoding strategies to instrument IDs, timestamps, prices, flags, and free-text fields.

Query plans, pruning, and pushdown

INTERMEDIATE

ONE-SENTENCE MEANING

A query plan converts declarative SQL into scans, filters, joins, aggregates, exchanges, and writes. Pruning and predicate pushdown reduce data early, while join order and distribution determine whether intermediate results remain bounded.

MEMORY JOGGER

Read the plan from leaves upward and ask where rows, columns, and bytes first shrink. The most expensive operator is often created by an earlier failure to filter, cluster, or estimate cardinality.

WHY IT MATTERS IN QUANT RESEARCH

A correct research query can still be operationally unusable if it scans every partition, broadcasts a huge table, or spills a many-to-many join. Plan literacy lets researchers distinguish data scale from avoidable execution mistakes.

FORMULA INTUITION

Selectivity is surviving rows divided by input rows, and cardinality estimates guide join strategy. Partition filters, column projection, row-group statistics, and runtime filters should reduce I/O before decompression and network shuffle where possible.

SIMPLE MARKET EXAMPLE

A ten-year feature query missing a session predicate scans all history before filtering in a Python client. Moving the predicate into SQL and joining a small universe dimension early can cut transferred bytes by orders of magnitude.

PYTHON / SYSTEM IMPLEMENTATION

Capture explain plans with runtime metrics, compare estimated versus actual rows, and monitor scan bytes, shuffle, spill, skew, and task tails. Parameterize dates and keys so filters remain visible to the optimizer.

CONFUSIONS TO AVOID

Adding a LIMIT does not necessarily reduce work if sorting or aggregation must complete first. Common table expressions, functions, or casts can also prevent pruning depending on engine behavior.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Query-plan evidence feeds physical design, materialization, workload management, and cost review. Stable production queries should have regression tests for both results and bounded resource behavior.

RELATED CONCEPTS AND QUICK RECALL

Related: predicate pushdown, cardinality estimate, shuffle, spill. Recall task: Inspect a slow point-in-time join and identify the first operator where data should have been reduced.

Point-in-time and as-of querying

INTERMEDIATE

ONE-SENTENCE MEANING

A point-in-time query returns the version of each fact or attribute that was effective and knowable at a decision cutoff. It combines stable identity, effective intervals, availability clocks, and deterministic tie-breaking rather than simply selecting the latest row.

MEMORY JOGGER

Historical truth means as known then, not the final corrected value now. Every as-of query should make its left-side decision time and right-side eligibility rule visible.

WHY IT MATTERS IN QUANT RESEARCH

Fundamentals, classifications, index membership, borrow, corporate actions, and corrected prices all change or arrive late. Current-state joins create look-ahead bias even when the model split itself is chronological.

FORMULA INTUITION

For decision i , choose record j with the greatest eligible availability time such that entity keys match, $\text{effective_from}_j \leq \text{decision}_i < \text{effective_to}_j$, and $\text{available}_j \leq \text{cutoff}_i$. Ties require source priority and revision rules.

SIMPLE MARKET EXAMPLE

A sector change effective Monday is published Tuesday. Monday's close decision cannot use it, while a Wednesday decision may if processing finished before its cutoff.

PYTHON / SYSTEM IMPLEMENTATION

Implement temporal interval constraints or a tested as-of operator, emit selected version IDs, and reconcile unmatched and multiple candidates. Build fixtures around boundary equality, late arrival, correction, deletion, and identity change.

CONFUSIONS TO AVOID

Joining on nearest timestamp can select future data unless direction and tolerance are explicit. Filtering after a latest-row window function can also leak because the ranking considered ineligible future versions.

WHERE THIS FITS IN THE RESEARCH SYSTEM

A shared temporal query service or library should serve universe, reference, feature, and cost pipelines. Artifacts record the query policy and source snapshot alongside the output.

RELATED CONCEPTS AND QUICK RECALL

Related: as-of join, bitemporal data, effective time, availability time. Recall task: Write the eligibility rules for a historical sector join and explain how ties are resolved.

Materialized views, caches, and projections

INTERMEDIATE

ONE-SENTENCE MEANING

A materialized view stores a derived query result, a cache reuses work under an identity key, and a projection stores an alternate physical arrangement of governed data. Each trades storage and maintenance for lower latency, so invalidation and freshness are part of

MEMORY JOGGER

Acceleration is safe only when the consumer can tell which source snapshot and policy the result represents. A fast stale answer is a data defect, not a performance win.

WHY IT MATTERS IN QUANT RESEARCH

Repeated universe filters, return aggregates, and latest-state queries can dominate research cost. Governed materialization reduces redundant computation while keeping source lineage and point-in-time semantics visible.

FORMULA INTUITION

Break-even depends on build cost B , per-query savings S , expected reuse n , maintenance M , and storage cost; materialize when nS exceeds $B + M$ under the freshness requirement. Cache keys must include source snapshot, normalized query or transform, parameters, schema, and policy.

SIMPLE MARKET EXAMPLE

A daily liquidity summary used by fifty factor jobs is materialized after canonical bars close. An intraday consumer uses a separate projection with a stricter refresh frontier rather than assuming the daily table is current.

PYTHON / SYSTEM IMPLEMENTATION

Store parent versions, refresh clock, row counts, checksums, and quality status with every derived result. Make refresh atomic and expose stale, building, failed, and ready states to readers.

CONFUSIONS TO AVOID

Refreshing on a schedule does not prove freshness if upstream data arrived late. Query-result caches can also cross user entitlements or parameter boundaries unless identity includes security context.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Acceleration artifacts sit between canonical storage and demanding consumers. The registry records ownership, refresh dependency, retention, and the models or reports that pin each version.

RELATED CONCEPTS AND QUICK RECALL

Related: materialized view, cache key, projection, invalidation. Recall task: Decide whether to materialize a daily liquidity summary and define its complete cache identity.

Concurrency and workload management

INTERMEDIATE

ONE-SENTENCE MEANING

Workload management allocates CPU, memory, I/O, queue capacity, and priority among competing queries and maintenance jobs. It protects decision-critical work from exploratory scans and prevents one malformed query from exhausting shared infrastructure.

MEMORY JOGGER

Concurrency is not free parallelism; every active task consumes memory, file handles, network, and scheduler attention. Admit work according to measured resource envelopes and business deadlines.

WHY IT MATTERS IN QUANT RESEARCH

Research backfills, interactive notebooks, scheduled features, data quality checks, and live risk reads often share storage. Without isolation, a large historical scan can delay a market-open dependency or trigger broad spilling.

FORMULA INTUITION

If each worker peaks at m GB and the pool has M usable GB, a first concurrency bound is $\text{floor}(M/m)$ before safety margin and skew. Queue latency plus execution latency must fit the consumer deadline, and tail percentiles matter more than mean.

SIMPLE MARKET EXAMPLE

Eight backfill queries each request 14 GB on a 96-GB pool with 20 GB reserved. Admitting all eight guarantees pressure, so the scheduler caps concurrency, lowers priority, or moves them to a separate pool.

PYTHON / SYSTEM IMPLEMENTATION

Classify workloads by deadline, resource profile, and preemption safety; set quotas, timeouts, spill limits, and queue policies. Capture query tags and lineage so expensive work can be traced to artifacts and owners.

CONFUSIONS TO AVOID

Increasing concurrency can reduce throughput when tasks compete for memory bandwidth or storage requests. Priority without admission control merely changes which workload causes the outage.

WHERE THIS FITS IN THE RESEARCH SYSTEM

The orchestration and query layers share resource classes and deadlines. Production publications, freshness checks, and live serving receive protected capacity while exploratory work remains bounded and observable.

RELATED CONCEPTS AND QUICK RECALL

Related: admission control, resource pool, query queue, tail latency. Recall task: Calculate a safe initial concurrency cap and define which quant workloads receive protected capacity.

Transactions and atomic publication

INTERMEDIATE

ONE-SENTENCE MEANING

A transaction groups changes into an all-or-nothing state transition, while atomic publication makes a complete validated dataset visible in one logical step. These controls stop readers from observing half-written partitions or inconsistent metadata.

MEMORY JOGGER

Write to staging, validate, commit a manifest, then move a governed pointer. Readers resolve one snapshot at job start and never assemble latest files independently.

WHY IT MATTERS IN QUANT RESEARCH

Backfills and corrections can span hundreds of objects, and concurrent researchers may query during publication. Atomic visibility preserves reproducible table state even when physical writes are distributed and long-running.

FORMULA INTUITION

ACID describes atomicity, consistency, isolation, and durability, but each system implements specific guarantees and failure modes. Dataset commits commonly use optimistic concurrency: publish only if the expected parent pointer is unchanged.

SIMPLE MARKET EXAMPLE

Two jobs compact the same quote partition. The first commits against snapshot 41; the second detects that the parent moved and must rebase or abort instead of silently dropping the first job's files.

PYTHON / SYSTEM IMPLEMENTATION

Use transaction logs or manifests, conditional commits, checksums, and recovery tests for failure before, during, and after commit. Record writer identity, parent snapshot, schema, and quality disposition.

CONFUSIONS TO AVOID

Renaming individual files is not an atomic dataset transaction, especially in object storage. A transaction label also does not prove serializable behavior, so isolation and conflict detection must be tested.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Transactions protect table metadata, reference updates, artifact registries, and serving pointers. Immutable source files plus atomic manifests give research consumers a defensible snapshot boundary.

RELATED CONCEPTS AND QUICK RECALL

Related: ACID, optimistic concurrency, manifest commit, isolation. Recall task: Describe the commit and recovery sequence for publishing a partitioned feature table.

Quality constraints and reconciliation

INTERMEDIATE

ONE-SENTENCE MEANING

Quality constraints encode structural, relational, temporal, and statistical expectations, while reconciliation compares independent counts or values across boundaries. Together they turn storage from a passive container into a controlled evidence system.

MEMORY JOGGER

Validate near the producer and again at high-risk consumption points. Preserve observed metrics and failure samples even on passing runs so gradual deterioration remains visible.

WHY IT MATTERS IN QUANT RESEARCH

Duplicate trades, missing sessions, stale partitions, unit shifts, and broken reference joins can all produce valid numeric outputs. Database-level and artifact-level checks catch these issues before they become model behavior.

FORMULA INTUITION

Core measures include unique-key ratio, observed-to-expected coverage, null and finite rates, freshness lag, join match classes, aggregate deltas, and distribution quantiles. Thresholds should be sliced by venue, asset class, source, and regime rather than only globally.

SIMPLE MARKET EXAMPLE

A daily bar load has the expected total row count but one venue is absent and another duplicated. Global volume appears close, while venue-level coverage and uniqueness reconciliation expose the defect.

PYTHON / SYSTEM IMPLEMENTATION

Attach constraints to schema versions, save check SQL and results, and block atomic publication on hard failures. Use independent source totals or accounting identities where possible rather than self-reconciliation.

CONFUSIONS TO AVOID

A passed check is not proof when the check shares the same flawed transformation as the output. Static thresholds can also normalize persistent degradation if baselines are updated automatically.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Quality reports become first-class parents of canonical, feature, model, and serving artifacts. Incident tools connect each failure to source snapshots, affected consumers, and repair actions.

RELATED CONCEPTS AND QUICK RECALL

Related: constraint, reconciliation, coverage, quarantine. Recall task: Design hard and soft checks for daily bars and identify an independent reconciliation source.

Backups, recovery, and disaster readiness

INTERMEDIATE

ONE-SENTENCE MEANING

Backups preserve recoverable copies, while disaster recovery defines how services and data resume after loss or corruption. Recovery point objective limits acceptable data loss, and recovery time objective limits acceptable service interruption.

MEMORY JOGGER

A backup is valuable only after a successful restore test. Protect metadata, schemas, catalogs, encryption keys, and manifests as carefully as bulk data files.

WHY IT MATTERS IN QUANT RESEARCH

Object durability does not protect against logical deletion, compromised credentials, corrupted commits, or region-wide failure. Quant evidence and model approvals need recovery paths that preserve version and lineage integrity.

FORMULA INTUITION

RPO is the maximum time between recoverable states and failure; RTO is the maximum time from incident to restored service. Backup frequency, replication lag, restore bandwidth, catalog rebuild time, and validation determine whether those objectives are credible.

SIMPLE MARKET EXAMPLE

Canonical files survive in object storage, but the table catalog and encryption key are lost. Without protected metadata and key recovery, durable bytes are operationally unusable and snapshots cannot be reconstructed safely.

PYTHON / SYSTEM IMPLEMENTATION

Use versioning or immutable backup targets, separate administrative credentials, replicate critical metadata, and run timed restore drills. Validate checksums, table snapshots, row counts, permissions, and consumer pointers after recovery.

CONFUSIONS TO AVOID

Replication is not a backup if destructive changes propagate immediately. A restore that returns bytes without catalog history, access controls, or documented snapshot selection is incomplete.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Disaster plans cover raw evidence, canonical tables, registries, secrets, schedulers, and consumer failover. Runbooks state degraded modes and which research or trading actions must stop.

RELATED CONCEPTS AND QUICK RECALL

Related: backup, RPO, RTO, disaster recovery. Recall task: Set RPO and RTO for raw events, canonical bars, and experiment metadata, then outline restore validation.

Retention, tiering, compaction, and vacuum

INTERMEDIATE

ONE-SENTENCE MEANING

Retention determines how long versions and data remain, tiering moves colder content to cheaper storage, compaction rewrites small files into efficient groups, and vacuum removes unreferenced physical objects. These policies must preserve legal, licensing, lineage, and

MEMORY JOGGER

Data age is not the same as dispensability. Before deletion, prove that no retained snapshot, model, investigation, or contract still references the object.

WHY IT MATTERS IN QUANT RESEARCH

Tick history can become expensive, small files can degrade every query, and indefinite snapshots can block cleanup. Lifecycle policy controls cost without turning old research into an unreproducible claim.

FORMULA INTUITION

Expected retained bytes depend on ingest rate, compression, retention horizon, replica count, and snapshot sharing. Vacuum safety requires a reachability set from all retained manifests plus a grace interval longer than the maximum reader and commit duration.

SIMPLE MARKET EXAMPLE

A compaction job rewrites 20,000 tiny quote files into 200 balanced files under a new snapshot. Vacuum deletes the old files only after all pinned experiments and rollback windows release the prior snapshot.

PYTHON / SYSTEM IMPLEMENTATION

Maintain retention classes by dataset and license, inventory references, schedule compaction from file-size evidence, and dry-run deletions. Record each lifecycle action as an auditable artifact with before-and-after counts and bytes.

CONFUSIONS TO AVOID

Aggressive vacuum can break long-running readers or historical artifacts, while never vacuuming creates uncontrolled cost and metadata burden. Cold tiering may also violate retrieval deadlines if restore latency is ignored.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Lifecycle services consult the lineage registry, legal policy, data entitlements, and workload SLOs. Cost forecasts and recovery plans use the same retention and tier assumptions.

RELATED CONCEPTS AND QUICK RECALL

Related: retention policy, storage tier, compaction, garbage collection. Recall task: Define when old quote files become eligible for compaction, cold tiering, and final deletion.

Security, entitlements, and data licensing

INTERMEDIATE

ONE-SENTENCE MEANING

Security controls who can read, write, administer, and export data, while entitlements enforce source and dataset-specific usage rights. Market-data licensing can restrict users, environments, derived outputs, redistribution, geography, and retention independently of technical

MEMORY JOGGER

Identity, purpose, dataset, environment, and action all belong in the access decision. Least privilege applies to service accounts and automated exports as much as to human users.

WHY IT MATTERS IN QUANT RESEARCH

Quant systems combine expensive licensed feeds, proprietary signals, portfolio data, and model artifacts. A technically correct pipeline can still create legal or operational risk by copying restricted fields into broadly accessible research tables.

FORMULA INTUITION

An authorization decision can be expressed as `allow(subject, resource, action, context)` under policy version `p`, with every decision logged. Derived-data policy must specify whether aggregation or transformation changes entitlement, rather than assuming it does.

SIMPLE MARKET EXAMPLE

A researcher may query delayed exchange data in development but cannot export raw records or use real-time fields on an unlicensed production host. A derived volatility feature may still inherit redistribution restrictions depending on contract language.

PYTHON / SYSTEM IMPLEMENTATION

Tag datasets and fields with source, sensitivity, license class, retention, and export rules; enforce policy in catalogs, query engines, object storage, and delivery paths. Rotate secrets, separate roles, and audit unusual scans and downloads.

CONFUSIONS TO AVOID

Network isolation alone is not authorization, and row-level security does not govern copied files after export. Hashing or aggregating data does not automatically remove contractual restrictions.

WHERE THIS FITS IN THE RESEARCH SYSTEM

Entitlement metadata travels from ingestion through canonical, serving, feature, model, and report artifacts. Promotion checks verify that deployment identities and environments possess every required right.

RELATED CONCEPTS AND QUICK RECALL

Related: least privilege, entitlement, data license, audit log. Recall task: Design an access decision for delayed research data and list how restrictions propagate to derived artifacts.

Storage architecture and truth boundaries

INTERMEDIATE

HOW TO READ IT

Each layer has a different truth and performance contract. Arrows carry immutable snapshot IDs, clock frontiers, quality status, and entitlements rather than only file locations.

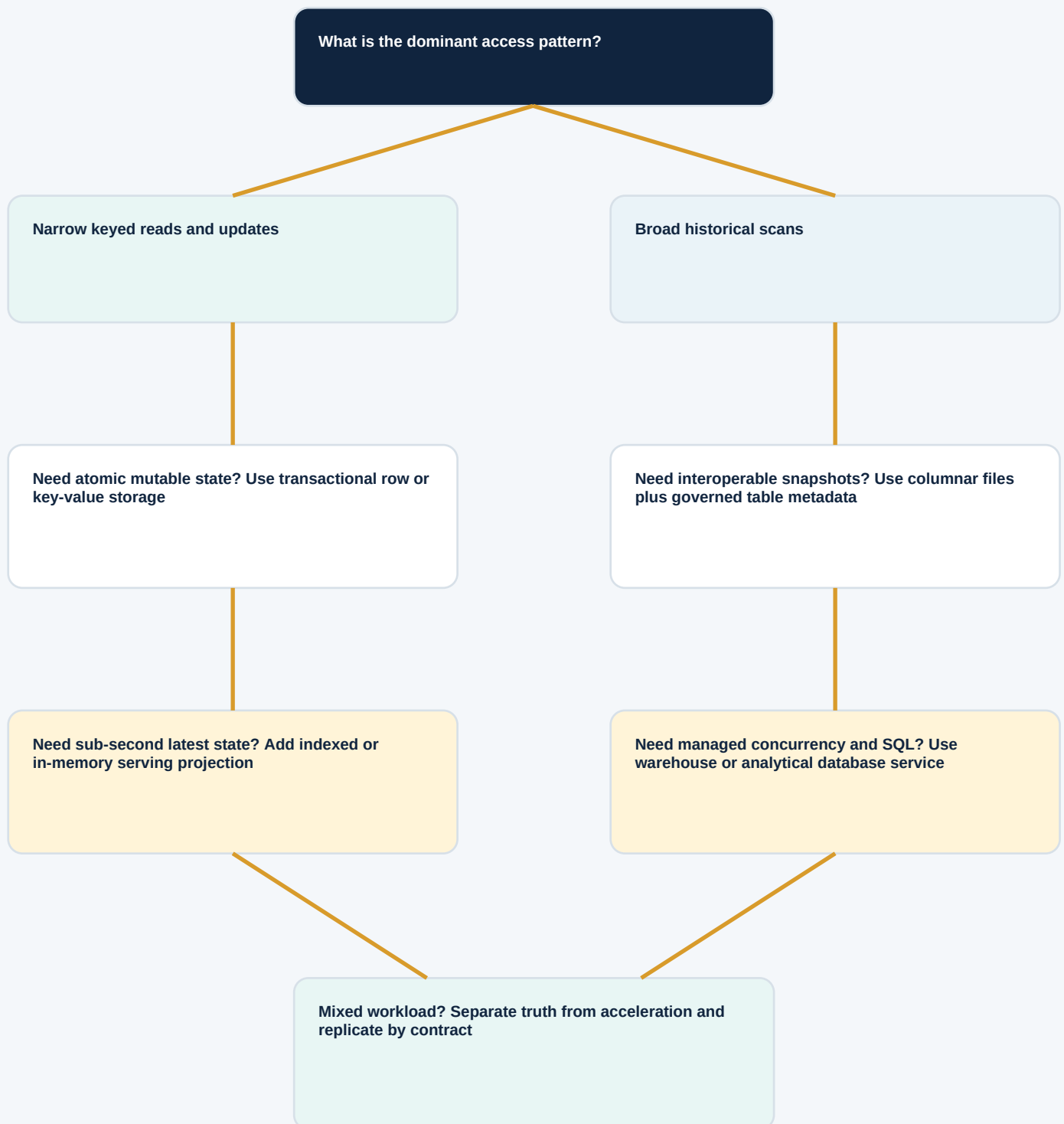


DESIGN PRINCIPLE

The canonical snapshot is the semantic handoff. Specialized serving layers may be rebuilt, but no consumer should have to guess which raw evidence, catalog state, or policy produced its data.

Choose storage from the workload

INTERMEDIATE



SELECTION TEST

Benchmark the chosen pattern at representative history, cardinality, concurrency, and failure conditions. Record the rejected alternatives and the workload change that would require revisiting the decision.

Worked grain and key design

INTERMEDIATE

SCENARIO

Design storage for raw exchange trades, accepted canonical trades, and one-minute bars. The tables share economic events but require different grains, keys, revision behavior, and completeness evidence.

TABLE	ONE ROW MEANS	KEY	CLOCKS	REVISION
Raw trade	One received source message	feed, channel, session, sequence	event, arrival	append; preserve corrections
Canonical trade	One accepted economic trade version	venue, trade ID, revision	event, available	new accepted version
Minute bar	One instrument-window summary	instrument, venue, minute	window, ready	new bar snapshot

1. RAW UNIQUENESS

Reject accidental duplicate transport messages by source key, but keep a corrected message with a distinct sequence or revision record.

2. CANONICAL ACCEPTANCE

Resolve cancellation and correction semantics, then expose exactly one accepted version per trade identity for each knowledge cutoff.

3. BAR MEMBERSHIP

Aggregate only accepted trades whose event times fall in the window and whose availability satisfies the bar-ready policy.

4. RECONCILIATION

Store event count, volume, first and last source sequence, missing-range flags, and parent snapshot so a bar can be audited.

5. BOUNDARY TEST

Place trades exactly at 10:00:00 and 10:01:00 to prove half-open window membership and prevent double counting.

ANSWER

The three tables should not share one convenient key. Each key is derived from its row meaning, and lineage connects raw messages to accepted trades and bars without collapsing their distinct revision semantics.

Storage and query cost model

INTERMEDIATE

SCENARIO

A canonical quote table ingests 1.8 TB compressed per month, retains 36 months, keeps one regional replica, and serves 1,200 analytical queries per month. Each query currently scans 420 GB after pruning.

FORMULA BLOCK

Retained bytes = ingest x months x replica factor. Monthly scan bytes = queries x average bytes scanned. Total operating cost also includes replication and query work.

Primary retained data **1.8 TB x 36 = 64.8 TB**

Before snapshots and metadata

With one replica **64.8 TB x 2 = 129.6 TB**

Replication doubles durable bytes

Monthly scanned data **1,200 x 0.42 TB = 504 TB**

Query work exceeds retained primary bytes

After 70% scan reduction **504 TB x 0.30 = 151.2 TB**

Clustering and projection save 352.8 TB/month

Compaction overhead **Rewrite 12 TB/month x 2 I/O = 24 TB**

Read plus write during maintenance

DECISION INSIGHT

The largest opportunity is reducing repeated scan bytes, not trimming a few percent from retained storage. Benchmark a clustered projection or materialized subset, then include build cost, freshness, cache invalidation, and additional snapshot retention before claiming savings.

Partition strategies by workload

INTERMEDIATE

	Cross-section	One symbol	Late repair	Metadata	Compression	Parallelism
Month only	STRONG	WEAK	WEAK	STRONG	STRONG	FAIR
Day only	STRONG	FAIR	STRONG	FAIR	FAIR	STRONG
Venue-hour	FAIR	FAIR	STRONG	FAIR	FAIR	STRONG
Symbol-day	WEAK	STRONG	STRONG	POOR	WEAK	FAIR
Month + clustered symbol	STRONG	STRONG	FAIR	STRONG	STRONG	STRONG
Session + instrument bucket	STRONG	STRONG	STRONG	FAIR	STRONG	STRONG

INTERPRETATION

Month plus clustered symbol provides a balanced daily-bar layout for broad and selective reads, while venue-hour or session-bucket designs better bound high-frequency repair. Scores are illustrative; real selection must use measured predicates, skew, file size, and object counts.

FAILURE SIGNAL

A design that looks strong per query can fail operationally if it produces millions of tiny partitions or concentrates one busy venue on a single worker. Track metadata time and task-tail distribution alongside scan bytes.

Point-in-time query anatomy

INTERMEDIATE

SCENARIO

A strategy decides at Wednesday 09:30 using classification data. Version B is effective Tuesday but arrives Wednesday 10:15, so the earlier decision must retain version A despite B's earlier effective date.



ELIGIBILITY PREDICATE

Match stable entity; require `effective_from <= decision_time < effective_to`; require `available_from <= cutoff`; then rank eligible candidates by availability, revision precedence, and source priority. Emit the chosen version and match reason with the result.

INCORRECT PATTERN

Rank every historical version by effective date, choose the latest, and filter availability afterward. The ranking can select B first and discard A, producing a missing or leaked match.

CORRECT SEQUENCE

Filter candidates by identity, effective interval, and availability before ranking. Version A remains eligible at 09:30, while B enters only decisions after 10:15.

BOUNDARY TEST

Create records exactly at `effective_to` and cutoff. Decide whether intervals are half-open and whether equality at availability is admissible, then encode those choices in tests.

AUDIT OUTPUT

Retain left key, selected version, source snapshot, effective interval, availability time, policy version, and unmatched reason. This evidence makes leakage review tractable.

ANSWER

The Wednesday 09:30 decision uses version A. Version B may drive a revised canonical snapshot and later decisions, but it cannot retroactively alter the earlier information set.

Compression ratio versus decode speed

INTERMEDIATE

SYNTHETIC BENCHMARK

Four lossless codec configurations are tested on the same quote partition. The fastest choice depends on whether the workload is storage-bound, network-bound, or CPU-bound; a single compression ratio cannot select the winner.



INTERPRETATION

The balanced configuration often minimizes end-to-end time because it cuts scan bytes without exhausting decode CPU. The dense and maximum settings may still win for cold archival tiers or network-constrained reads, while the fast codec suits hot repeated scans.

BENCHMARK RULE

Test actual columns, sorting, row-group size, selectivity, concurrency, and cache state. Report logical throughput, physical bytes, CPU, memory, and p95 query time; synthetic points illustrate tradeoffs and are not product claims.

Read a query plan from the leaves

INTERMEDIATE

RESEARCH QUERY

Compute twenty-day average spread for the eligible universe, then join classifications and rank cross-sectionally. The plan below shows where bytes and rows should shrink before expensive exchange and aggregation.

1. Partition scan

12.0 TB -> 680 GB

Session predicate and four-column projection should prune first.

2. Runtime universe filter

680 GB -> 210 GB

A small eligible-key set removes nonmembers before the window.

3. Window aggregate

210 GB -> 48 GB

Maintain instrument ordering; watch repartition and spill.

4. Temporal classification join

48 GB -> 51 GB

Interval and availability predicates must precede candidate ranking.

5. Cross-sectional rank

51 GB -> 6 GB

Partition by decision time and keep stable sample identifiers.

6. Artifact write

6 GB + manifest

Publish data, counts, plan metrics, parents, and quality status atomically.

DIAGNOSTIC ORDER

If the plan is slow, locate the first stage whose actual rows, bytes, or distribution differ from expectation. Fix pruning, statistics, join cardinality, or skew there before tuning downstream operators.

Atomic publication and crash recovery

INTERMEDIATE

GOAL

A reader sees either snapshot 41 or complete snapshot 42, never a mixture. Data files may be written over hours, but visibility changes only when a validated manifest commits successfully.

1

PREPARE

Resolve parent 41, allocate run ID, write intent and expected schema.

2

WRITE

Create immutable data and delete files in staging; never mutate files from 41.

3

VALIDATE

Check keys, schema, counts, checksums, quality, entitlements, and completeness.

4

COMMIT

Conditionally append manifest 42 only if current pointer still equals 41.

5

PUBLISH

Move discovery pointer to 42 and emit readiness event with audit evidence.

6

CLEAN

Remove abandoned staging after grace period; retain 41 under rollback policy.

CRASH RULES

Before commit, unreachable staging may be retried or collected. After commit, manifest 42 is durable even if pointer notification fails; recovery reconciles the pointer from the transaction log instead of rewriting data blindly.

Recovery objectives by data class

INTERMEDIATE

DATA CLASS	RPO	RTO	RECOVERY MECHANISM	RESTORE VALIDATION
Raw exchange events	Near-zero	4 h	Replicated immutable objects + source replay	Sequence and checksum reconciliation
Canonical bars	15 min	2 h	Snapshot log + regional replica	Pinned snapshot, counts, returns continuity
Reference history	1 h	2 h	Versioned DB backup + change log	Interval overlap and point-in-time fixtures
Experiment registry	5 min	30 min	Transactional backup + log shipping	Artifacts, approvals, owners, permissions
Serving cache	24 h	15 min	Rebuild from canonical snapshot	Freshness and key parity before traffic
Model package store	Near-zero	1 h	Immutable replica + catalog backup	Signatures, dependencies, entitlements

REVIEW QUESTION

Are these objectives supported by measured backup frequency, replication lag, restore bandwidth, catalog recovery, key access, and validation time? A nominal target without a timed end-to-end drill is an aspiration, not a control.

Entitlement propagation

INTERMEDIATE

POLICY QUESTION

May this subject perform this action on this dataset in this environment for this purpose, and may the result leave the controlled boundary?
The answer depends on both security policy and source license.

SOURCE CONTRACT User classes, hosts, regions, retention, redistribution, derived-data terms

DATA TAGS Source, license class, sensitivity, field restrictions, allowed environments

IDENTITY Human or service account, team, role, device, session and approval context

POLICY DECISION Resource + action + purpose + environment under a versioned rule set

ENFORCEMENT Catalog, SQL engine, object store, API, export and model-promotion gates

AUDIT Decision, rows or bytes, destination, snapshot, policy version and anomaly signal

DERIVED-DATA WARNING

Aggregation, normalization, or model transformation does not automatically remove licensing obligations. Every export and deployment needs a documented inheritance or declassification rule supported by the governing contract.

Safe storage migration

INTERMEDIATE

1

DEFINE

Freeze source contract, workload baseline, target schema, rollback criteria

2

BACKFILL

Write target snapshots from immutable parents; preserve partition checkpoints

3

RECONCILE

Compare keys, clocks, values, distributions, query plans and entitlements

4

DUAL READ

Shadow real consumers and compare exact outputs plus readiness timing

5

CANARY

Move bounded users or dates; monitor errors, cost, latency and freshness

6

CUTOVER

Move governed pointer; retain source write and rollback compatibility

7

OBSERVE

Hold rollback window; close gaps; document performance and incidents

8

RETIRE

Remove old path only after lineage, retention and recovery approval

ROLLBACK STANDARD

Rollback requires more than restoring old bytes. Preserve compatible schemas, source write path, catalog state, access policy, and consumer pointer so the prior version can resume without losing changes made during the canary.

Storage failure diagnostics

INTERMEDIATE

SYMPTOM	LIKELY CAUSE	FIRST EVIDENCE	CONTROL ACTION
High scan bytes	Pruning absent or filters hidden	Plan scan nodes and partition predicates	Rewrite predicate; cluster or project
Many tiny tasks	Small files or fine partitions	File counts and bytes per task	Compact and coarsen partition spec
Long task tail	Skewed key or partition	Per-task rows, spill, and hot keys	Salt, rebalance, or isolate heavy keys
Wrong historical join	Availability filtered too late	Selected version and cutoff evidence	Filter eligible candidates before rank
Stale materialization	Upstream frontier not propagated	Parent snapshot and refresh state	Invalidate and republish atomically
Restore fails	Catalog or key missing	End-to-end drill evidence	Protect metadata and key recovery
Cost jumps	Retention, egress, or cache miss change	Cost by dataset, query tag, and tier	Fix identity, lifecycle, or projection
Access leak	Export bypasses policy boundary	Audit destination and policy version	Revoke, contain, rotate, and redesign

TRIAGE RULE

Find the earliest boundary where observed keys, clocks, bytes, or policy diverge from the contract. Treat downstream symptoms as impact evidence, not as permission to patch the final table in place.

Database observability scorecard

INTERMEDIATE

FRESHNESS

Measure source frontier, canonical snapshot readiness, materialization age, replication lag, and consumer deadline slack. Report by critical market and dataset rather than only globally.

COMPLETENESS

Compare stable keys, sessions, event sequences, files, partitions, and expected populations. Preserve denominators and slice gaps by venue, source, asset class, and revision state.

CORRECTNESS

Track constraint failures, reconciliation deltas, temporal violations, schema drift, duplicate keys, and snapshot conflicts. Link each metric to the exact policy and table version.

PERFORMANCE

Observe scanned bytes, pruning, cache hits, query latency percentiles, queue time, shuffle, spill, skew, decode CPU, and file-open overhead. Baseline by query class.

STORAGE HEALTH

Monitor retained bytes, object count, file-size distribution, compaction backlog, unreferenced data, hot-tier growth, and retrieval latency from cold tiers.

RELIABILITY

Measure successful commits, aborted conflicts, failed refreshes, backup age, restore drill duration, regional replication state, and consumer-visible downtime.

SECURITY

Audit denied and allowed access, unusual scan or export volume, privileged actions, policy drift, secret age, and entitlement mismatches across environments.

CONSUMER IMPACT

Map stale, corrupt, unavailable, or restricted snapshots to features, models, portfolios, reports, and capital. Prioritize by affected decision rather than log count.

RESPONSE

Every threshold has an owner, runbook, stop condition, degraded path, recovery artifact, and escalation clock. Review exceptions and error-budget burn on a fixed cadence.

Storage design promotion checklist

INTERMEDIATE

WORKLOAD CONTRACT

Representative reads, writes, history, freshness, revisions, concurrency, retention, and recovery objectives are quantified and benchmarked.

SEMANTIC CONTRACT

Every table declares grain, keys, units, clocks, source identity, null states, revision policy, ownership, and point-in-time eligibility.

PHYSICAL DESIGN

Partitioning, clustering, sorting, indexes, row groups, file sizes, compression, and compaction are justified by measured access patterns.

SNAPSHOT BEHAVIOR

Atomic commits, conflict detection, time travel, concurrent readers, rollback, garbage collection, and partial-failure recovery have passed tests.

QUERY EVIDENCE

Plans show bounded scans, correct cardinality, controlled shuffle and spill, stable tail latency, and no temporal filtering after candidate ranking.

QUALITY

Schema, relational, temporal, coverage, statistical, and independent reconciliation checks publish evidence and block unsafe snapshots.

RESILIENCE

Backups include data, manifests, catalogs, schemas, policies, and keys; timed restoration meets defined RPO and RTO with validation.

LIFECYCLE

Retention, tiering, compaction, vacuum, migration, and deprecation preserve pinned lineage, legal holds, reader grace periods, and cost visibility.

SECURITY AND LICENSE

Least privilege, field and dataset tags, environment restrictions, export controls, derived-data rules, audit, and incident response are enforced end to end.

Final active recall and system index

INTERMEDIATE

START FROM WORKLOAD

State the decision, query shape, history, freshness, concurrency, revision, retention, and failure tolerance before selecting technology.

DEFINE STORED MEANING

Write grain, stable and source keys, units, clocks, null states, revision rules, and entitlements. No physical optimization can repair ambiguous semantics.

PRESERVE EVIDENCE

Keep raw inputs immutable, publish canonical snapshots atomically, and pin all approved research. Mutable latest pointers are discovery aids only.

MODEL TIME HONESTLY

Filter identity, effective interval, and availability before ranking versions. Test equality boundaries, late corrections, deletions, and symbol changes.

ENGINEER PHYSICAL LAYOUT

Use measured predicates to choose partitions, clustering, row groups, indexes, and compression. Track scan and metadata amplification together.

READ PLANS AND TAILS

Locate the first operator where bytes or rows fail to shrink. Watch estimates, shuffle, spill, skew, queue time, and p95 or p99 latency.

ACCELERATE WITH LINEAGE

Materializations, caches, and projections include parent snapshot, parameters, policy, schema, security context, freshness, and quality state.

RECOVER THE WHOLE SYSTEM

Protect files, manifests, catalogs, schemas, policies, and keys. Prove RPO and RTO through timed restores and consumer validation.

GOVERN THE LIFECYCLE

Retention, tiering, compaction, vacuum, migration, and export must honor pinned lineage, contracts, licensing, and rollback windows.

FINAL STANDARD

Reject any database design whose historical truth, query behavior, recovery path, cost envelope, or entitlement decision cannot be demonstrated from durable evidence.